The Unreasonable Effectiveness of an LLM Agent Loop with Tool Use 工具使用下 LLM 智能体循环的惊人效果

Sketch.dev 2:18 分钟阅读 483 字 EN

2025-05-15 by Philip Zeyliger

2025-05-15 作者：Philip Zeyliger

My co-workers and I have been working on an AI Programming Assistant called

我和同事们过去几个月一直在开发一款名为 Sketch for the last few months. The

thing I've been most surprised by is how shockingly simple the main loop of using an LLM with tool use is:

的 AI 编程助手。最让我惊讶的是，使用 LLM 结合工具调用的主循环竟简单得令人震惊：

```
def loop(llm):
    msg = user_input()
    while True:
        output, tool_calls = llm(msg)
        print("Agent: ", output)
        if tool_calls:
            msg = [ handle_tool_call(tc) for tc in tool_calls ]
        else:
            msg = user_input()
```

There's some pomp and circumstance to make the above work (

为了让上述流程顺利运行需要做些准备工作（here's the full script

这是完整脚本) , but the core idea is the above 9 lines. Here, `llm()` is a function

that sends the system prompt, the conversation so far, and the next message to the LLM API.

但核心思想就是上面这 9 行代码。这里的 `llm()` 是一个函数，负责将系统提示、当前对话内容和下一条消息发送给 LLM 接口。

Tool use is the fancy term for "the LLM returns some output that corresponds to a schema," and, in the full script, we tell the LLM (in its system prompt and tool

command line options, it adapts. (It can also be infuriating! "Oh, this test doesn't pass... let's just skip it," it sometimes says, maddeningly.)

仅凭这一款通用工具，当前模型（我们大量使用 Claude 3.7 Sonnet）就能解决许多问题，有些甚至能"一蹴而就"。过去我需要查阅生僻的 git 操作然后复制粘贴，现在只需让 Sketch 代劳；曾经手动处理 git 合并的日子，如今由 Sketch 打头阵；从前修改类型后要逐条排查类型检查错误（老实说有时还会陷入 `perl -pie` 的混乱局面），现在交给 Sketch 尝试。只要提示得当，这个智能代理循环就能持续运作。若缺少某些工具，它会自动安装；遇到不同命令行选项的`grep`命令，它也能灵活适应。（当然有时也让人抓狂！它会冷不丁冒出一句："哦，这个测试没通过...干脆跳过吧"，简直令人火大。）

For many workflows, agentic tools specialize. Sketch's quiver of tools is not just `bash`, as we've found that a handful of extra tools improve the quality, speed up iterations, and facilitate better developer workflows. Tools that let the LLM edit text correctly are surprisingly tricky. Seeing the LLM struggle with `sed` one-liners re-affirms that visual (as opposed to line) editors are a marvel.

对于许多工作流程而言，智能工具需要专业化。Sketch 的工具库并非只是 `bash` ，因为我们发现少量额外工具能提升质量、加速迭代并优化开发者工作流。让 LLM 正确编辑文本的工具出人意料地复杂。目睹 LLM 在 `sed` 单行代码上的挣扎，再次印证了可视化（而非行式）编辑器堪称奇迹。

I have no doubt that agent loops will get incorporated into more day to day automation tedium that's historically been too specific for general purpose tools and too esoteric and unstable to automate traditionally. I keep thinking of how much time I've spent correlating stack traces with git commits, and how good LLMs are at doing a first pass on it. We'll be seeing more custom, *ad hoc*, throw-away LLM agent loops in our `bin/` directories. Grab your favorite bearer token and give it a shot.

我毫不怀疑智能循环将被融入更多日常自动化琐事中——这些任务过去因过于特定而无法使用通用工具，又因过于晦涩不稳定而难以传统自动化。我不断回想自己曾花费多少时间将堆栈跟踪与 git 提交记录关联，而 LLM 在此类初步处理上表现如此出色。我们将看到更多定制化、临时性、一次性的 LLM 智能循环出现在 `bin/` 目录中。拿起你最爱的身份令牌，亲自试试吧。

Also published at philz.dev/blog/agent-loop/.

本文亦发布于 philz.dev/blog/agent-loop/

sketch.dev · merde.ai · pi.dev